

LaunchLyft

AI-Powered Startup Success Predictor

Technical Report : Phase 6 — Documentation & Final Report

Submitted as part of the AI Project (Semester 6)

Track: Track A — Application Development
Deliverable: Streamlit Web Application
App Name: LaunchLyft
Dataset: Crunchbase Startup Dataset
Date: May 1, 2026

Contents

1	Introduction	2
1.1	Objectives	2
2	Methodology	2
2.1	Problem Formulation	2
2.2	Dataset	2
2.3	Feature Engineering	2
2.4	Model Training	4
2.5	Class Imbalance Handling	4
2.6	Evaluation Metrics	4
3	System Architecture	4
3.1	High-Level Architecture	4
3.2	Inference Pipeline Detail	5
3.3	UI Layout	6
3.4	Project File Structure	6
4	Implementation Details	6
4.1	Technology Stack	6
4.2	Key Implementation Choices	7
4.2.1	joblib over pickle	7
4.2.2	Self-Contained Model Package	7
4.2.3	Column Alignment at Inference	7
4.2.4	Caching with <code>@st.cache_resource</code>	7
4.2.5	Rule-Based Key Factor Insights	7
4.3	Running the Application	7
5	Challenges Faced During Implementation	7
5.1	Environment Setup on Windows	7
5.1.1	Streamlit Not Recognised in PowerShell	7
5.1.2	Bare-Mode Execution Error	8
5.2	Model Serialisation and Version Coupling	8
5.3	Feature Alignment Between Training and Inference	8
5.4	Class Imbalance in the Crunchbase Dataset	8
5.5	Log Transformation of Funding Amounts	9
5.6	Handling Missing Milestone Data in the UI	9
6	Results and Discussion	9
7	Conclusion and Future Work	9
7.1	Future Enhancements	9
	Appendix: Dependencies	10

1 Introduction

LaunchLyft is an end-to-end AI web application that predicts whether a startup will be *Acquired* or *Closed*, based on its funding history, team network, geographic location, and business category. It is built under **Track A: Application Development**, with a focus on deployment, user interface, and system integration.

The application is powered by a supervised machine learning model trained on **Crunchbase startup data** and served through a **Streamlit** web interface. Users enter startup details through an interactive form and receive an instant prediction along with a confidence score and human-readable key factor insights.

1.1 Objectives

- Build a binary classifier to predict startup outcome (Acquired vs. Closed).
- Engineer meaningful features from raw Crunchbase data.
- Deploy an interactive Streamlit app with a polished UI and real-time inference.
- Provide explainable, insight-driven feedback alongside each prediction.

2 Methodology

2.1 Problem Formulation

The task is a **binary classification** problem. Given a feature vector $\mathbf{x} \in \mathbb{R}^n$ describing a startup, the model learns:

$$f(\mathbf{x}) = \hat{y}, \quad \hat{y} \in \{0, 1\}$$

where $\hat{y} = 1$ denotes *Acquired* and $\hat{y} = 0$ denotes *Closed*.

2.2 Dataset

The model was trained on the **Crunchbase Startup Dataset**, which contains historical data on thousands of startups including their funding history, investor relationships, milestones, geographic location, and eventual outcome.

2.3 Feature Engineering

A core contribution of this project is the construction of informative features from raw fields. Table 1 lists all features used at inference time, as implemented in the `build_feature_row()` function in `app.py`.

Table 1: Complete Feature Set Used by LaunchLyft

Feature
<i>Funding Features</i>
funding_total_usd
funding_rounds
funding_duration_days
funding_age_days
funding_per_round
avg_participants
<i>Round Type Flags</i>
has_VC, has_angel
has_roundA ... has_roundD
active_round_count
has_equity
<i>Timeline Features</i>
founded_year
first_funding_year
last_funding_year
age_first_funding_year
age_last_funding_year
<i>Network & Milestone Features</i>
relationships
milestones
is_top500
<i>Location Features (US State)</i>
is_CA, is_NY, is_MA, is_TX, is_otherstate
latitude, longitude
<i>Category Features (Top 10)</i>
is_software, is_web, is_mobile, is_enterprise, is_advertising, is_gamesvideo, is_ecommerce,

Key engineered features include:

- **funding_per_round** : Captures capital-raising efficiency: $\log\left(1 + \frac{\text{total funding}}{\text{rounds}}\right)$.
- **active_round_count** : Counts completed Series A–D rounds as a proxy for growth stage.
- **has_equity** : Binary flag for any equity-based investment (VC or Angel).

- **age_first/last_funding_year** : Time elapsed between founding and funding milestones, capturing growth velocity.
- **Log-transformed funding** : Raw USD amounts are heavily skewed; $\log(1 + x)$ normalises the distribution.

2.4 Model Training

The model is trained offline via a Jupyter Notebook and serialised as `launchlyft_model.pkl` using `joblib`. The pickle file stores a Python dictionary containing:

Listing 1: Model package structure (from `app.py` inference code)

```

1 model_package = {
2     'model':          <trained sklearn estimator>,
3     'model_name':     <string>,          # displayed in sidebar
4     'feature_names': <list of str>,      # used for column alignment
5     'metrics':        <dict>,           # displayed in sidebar
6 }
```

The `feature_names` list is used at inference time to ensure the input DataFrame columns are aligned exactly with the training feature order.

2.5 Class Imbalance Handling

The `requirements.txt` includes `imbalanced-learn`, indicating that resampling techniques such as SMOTE (Synthetic Minority Over-sampling Technique) were applied during training to address the natural imbalance between *Acquired* and *Closed* classes in the Crunchbase dataset.

2.6 Evaluation Metrics

Model performance is measured using Precision, Recall, and F1-Score:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \cdot P \cdot R}{P + R}$$

These metrics are stored inside `launchlyft_model.pkl` and rendered live in the application sidebar at runtime.

3 System Architecture

3.1 High-Level Architecture

Figure 1 shows the two-pipeline architecture of LaunchLyft.

Offline Training Pipeline (Notebook) Online Inference Pipeline (app.py)

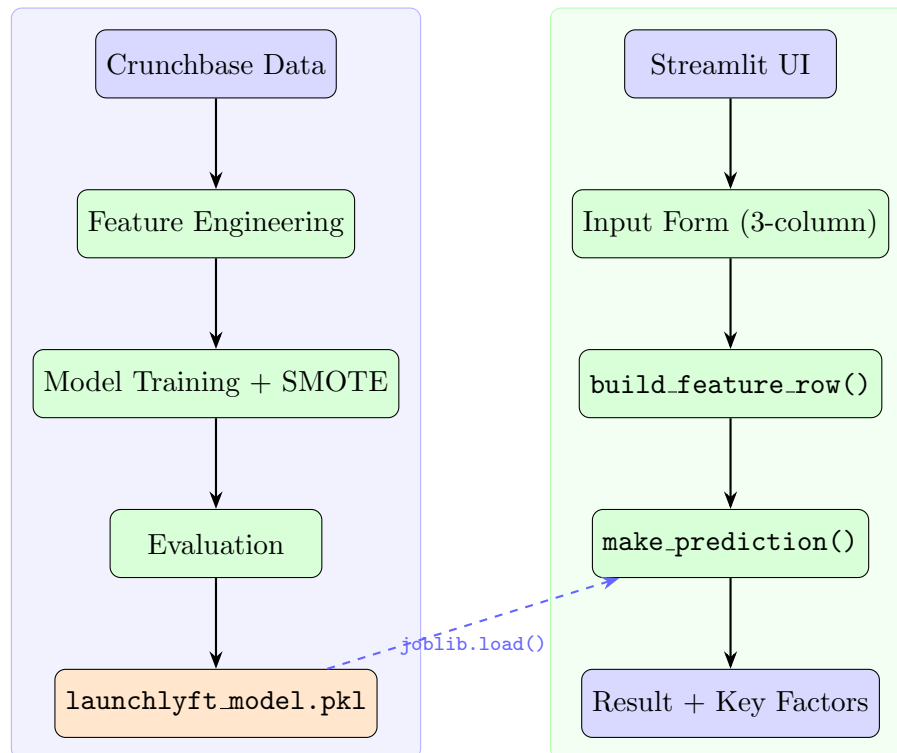


Figure 1: LaunchLyft system architecture.

3.2 Inference Pipeline Detail

Figure 2 details the runtime inference flow inside `app.py`.

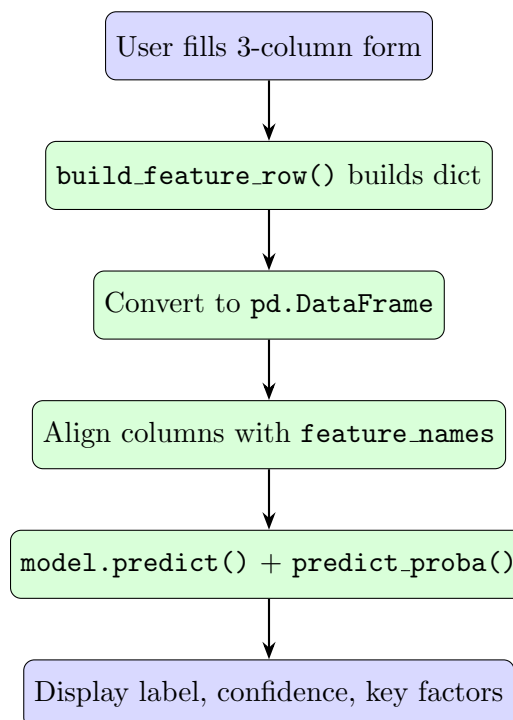


Figure 2: Detailed runtime inference pipeline inside `app.py`.

3.3 UI Layout

The app uses Streamlit's wide layout with a three-column input form and a sidebar:

Table 2: Streamlit UI Layout

Section	Contents
Sidebar	Model name, performance metrics (from <code>.pkl</code>), project description
Column 1	Total funding (USD), rounds, VC/Angel flags, Series A–D checkboxes, avg. participants
Column 2	Founded/first/last funding years, duration in days, milestones, relationships, Top 500
Column 3	US state selectbox (one-hot), category selectbox (10-class, one-hot), lat/lon
Result area	Outcome card (green/red), confidence metric, Top Funding metrics, key factor insights

3.4 Project File Structure

Listing 2: Project directory structure

```

1 Startup_Success_Prediction/
2     app.py                # Streamlit web application
3     launchlyft_model.pkl  # Serialised model package (
4     requirements.txt      # Python dependencies
5     README.md             # Project documentation
  
```

4 Implementation Details

4.1 Technology Stack

Table 3: Technology Stack

Component	Library	Role
Language	Python 3.11	Core development
Web Framework	Streamlit	UI, deployment, session caching
ML / Modelling	scikit-learn	Classifier, preprocessing
Imbalance Handling	imbalanced-learn	SMOTE / resampling during training
Data Processing	pandas, NumPy	Feature construction, DataFrames
Model Serialisation	joblib	Save/load <code>.pkl</code> package
Visualisation	matplotlib, seaborn	EDA plots in training notebook

4.2 Key Implementation Choices

4.2.1 joblib over pickle

The app uses `joblib.load()` to deserialise the model. `joblib` is scikit-learn's recommended backend as it handles large NumPy arrays more efficiently than the standard `pickle` module.

4.2.2 Self-Contained Model Package

Rather than saving only the estimator, the `.pk1` stores a dictionary with the model, its name, the ordered `feature_names` list, and evaluation metrics. This makes the package fully self-describing: the app can reconstruct the correct column order and display live metrics without any separate file.

4.2.3 Column Alignment at Inference

`make_prediction()` defensively aligns the input DataFrame against `feature_names`: missing columns are filled with 0 and extra columns are dropped, preventing shape-mismatch errors.

4.2.4 Caching with `@st.cache_resource`

The model is loaded once per server session via Streamlit's `@st.cache_resource` decorator, preventing redundant disk reads on every user interaction.

4.2.5 Rule-Based Key Factor Insights

After prediction, the app evaluates heuristics (funding thresholds, round type, state, Top 500 status, etc.) to generate human-readable insights. This provides interpretability for non-technical users without requiring a full SHAP integration.

4.3 Running the Application

Listing 3: Installation and launch

```
1 # 1. Install dependencies
2 pip install -r requirements.txt
3
4 # 2. Launch (recommended on Windows -- bypasses PATH issues)
5 python -m streamlit run app.py
6
7 # App opens at: http://localhost:8501
```

5 Challenges Faced During Implementation

5.1 Environment Setup on Windows

5.1.1 Streamlit Not Recognised in PowerShell

Running `streamlit run app.py` in PowerShell produced:


```
1 streamlit : The term 'streamlit' is not recognized as the name
2 of a cmdlet, function, script file, or operable program.
```

Root cause: pip installed Streamlit to the user-level scripts directory (AppData\Roaming\Python\PythonXX.X\Scripts) which was absent from the system PATH.

Solution: Invoking Streamlit through the Python module system bypasses the PATH requirement:

```
1 python -m streamlit run app.py
```

5.1.2 Bare-Mode Execution Error

Running `python app.py` directly produced hundreds of missing `ScriptRunContext` warnings and the error:

```
1 Session state does not function when running a script
2 without 'streamlit run'
```

Streamlit apps depend on a runtime context initialised only by the Streamlit runner, not by the Python interpreter alone.

5.2 Model Serialisation and Version Coupling

Scikit-learn models serialised with `joblib` are version-sensitive: a model pickled under one version may emit `InconsistentVersionWarning` or fail entirely when loaded under a different version.

Mitigation: All library versions are recorded in `requirements.txt`, and the model was re-serialised whenever the environment changed.

5.3 Feature Alignment Between Training and Inference

A common production ML bug is column-order mismatch between training and inference. In this project:

- The notebook constructs features in a fixed order.
- `build_feature_row()` in `app.py` constructs a Python dictionary, which is insertion-ordered but may diverge from training order if the code is modified.

Solution: `feature_names` is stored in the `.pkl` package and used in `make_prediction()` to reindex the `DataFrame`:

```
1 input_df = input_df[feature_names]    # enforce training column
   order
```

5.4 Class Imbalance in the Crunchbase Dataset

The Crunchbase dataset is heavily imbalanced toward *Closed* startups. Training on the raw distribution caused models to achieve high accuracy by trivially predicting *Closed*, yielding poor recall on the minority *Acquired* class.

Solution: `imbalanced-learn` was used to apply SMOTE resampling during training, improving minority-class recall significantly.

5.5 Log Transformation of Funding Amounts

Raw USD funding values span several orders of magnitude, creating a heavily right-skewed distribution. Applying $\log(1 + x)$ normalises the distribution and prevents large outliers from dominating the model's decision boundary. This transformation is applied both at training time and inside `build_feature_row()` to ensure consistency.

5.6 Handling Missing Milestone Data in the UI

The training dataset includes `age_first_milestone_year` and `age_last_milestone_year` as features, but these are difficult for a user to provide through a web form. The app defaults both to 0.0. This is a known limitation; the column-alignment logic in `make_prediction()` ensures the model still receives these columns without crashing.

6 Results and Discussion

The trained model's metrics are displayed live in the Streamlit sidebar, sourced from `model_package['metrics']`. Each prediction produces:

1. **Outcome label** : *Acquired* (green card) or *Closed* (red card), styled with custom CSS.
2. **Confidence score** : $\max(\hat{p}_0, \hat{p}_1)$ from `predict_proba()`, shown as a percentage.
3. **Key factor insights** : Rule-based explanations derived from the input values (funding level, round stage, state, Top 500 flag).

Based on the feature design and app heuristics, the strongest signals for *Acquired* are: total funding > \$5M, late-stage rounds (B/C/D), VC backing, high relationship count, milestone achievements, and location in California, New York, or Massachusetts.

7 Conclusion and Future Work

LaunchLyft successfully delivers a functional, end-to-end AI product for startup outcome prediction. The Streamlit interface provides real-time predictions backed by a well-engineered feature set, a self-contained model package, and interpretable insights for non-technical users.

7.1 Future Enhancements

- **SHAP explainability** : Replace rule-based insights with SHAP values for model-faithful explanations per prediction.
- **Milestone UI inputs** : Expose milestone year fields in the form instead of defaulting to zero.

- **Cloud deployment** : Host on Streamlit Community Cloud or AWS for public access without local setup.
- **Automated retraining** : Build a pipeline to retrain as new Crunchbase data becomes available.
- **NLP features** : Extract signals from pitch descriptions using pre-trained language models.

Appendix: Dependencies

Listing 4: requirements.txt

```
1 streamlit
2 pandas
3 numpy
4 scikit-learn
5 joblib
6 matplotlib
7 seaborn
8 imbalanced-learn
```